

Bubble Sort

O algoritmo compara pares de elementos vizinhos do início ao fim da lista: se o elemento da esquerda for maior que o da direita, eles trocam de lugar. Ele avança um passo e repete a comparação para o próximo par. Ao final da primeira passagem completa, o maior elemento da lista com certeza estará travado na última posição. O processo se repete para o restante da lista até que nenhuma troca seja necessária.

Bubble Sort Implementações:

```
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        bool trocou = false;
        // A cada passo 'i', os últimos 'i' elementos já estão no lugar certo
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;

                trocou = true;
            }
        }

        if (!trocou) {
            break;
        }
    }
}
```

Selection Sort

O Selection Sort é um algoritmo simples que funciona encontrando o menor elemento da parte ainda não ordenada do vetor e colocando-o na posição correta.

A ideia é:

Procure o menor elemento do vetor, troque esse elemento com a primeira posição. Agora considere que a primeira posição já está ordenada procure o menor elemento do restante do vetor e repita até o final.

O algoritmo possui complexidade: $O(n^2)$

```
void selectionSort(int v[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int menor = i;
```

```
// Procura o menor elemento
for (int j = i + 1; j < n; j++)
{
    if (v[j] < v[menor])
        menor = j;
}

// Troca os elementos
int aux = v[i];
v[i] = v[menor];
v[menor] = aux;
}
}
```

Insertion Sort

O Insertion Sort é um algoritmo de ordenação bem intuitivo.

Ele funciona da mesma forma que a maioria das pessoas usa para ordenar cartas em um jogo de baralho: você pega uma carta por vez e a insere na posição correta dentro do grupo de cartas que já estão ordenadas na sua mão.

O vetor é dividido mentalmente em duas partes: uma sublista ordenada (à esquerda) e uma sublista não ordenada (à direita).

Começamos considerando o primeiro elemento (índice 0) como uma lista já ordenada de tamanho. Pegamos o próximo elemento não ordenado (chamado de chave ou key), comparamos a chave com os elementos da esquerda (a parte ordenada), movendo para a direita todos os elementos que forem maiores que ela, inserimos a chave no espaço vago e repetimos até o final do vetor.

O algoritmo possui complexidade: $O(n^2)$

```
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int chave = arr[i]; // Elemento que vamos inserir na parte ordenada
        int j = i - 1;

        // Move os elementos de arr[0..i-1] que são maiores que a chave para uma
        // posição à frente de sua posição atual
        while (j >= 0 && arr[j] > chave) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }

        arr[j + 1] = chave;
    }
}
```